# INFORME DE LABORATORIO

**Autor:** *Sergio Andrés Alarcón López, Cristian Florez Grisales, Daniel Eduardo López Rodríguez*

*Laboratorio de Electrónica Digital 3*
*Departamento de Ingeniería Electrónica y de Telecomunicaciones*
*Universidad de Antioquia*

**Octavia**

Este documento detalla la implementación en C del sistema embebido Octavia, el cual es un instrumento musical. Se hace el desarrollo de drivers reutilizables del temporizador SysTick a partir de sus registros, Se busca implementar temporización, procesamiento de muestras y conversión de digital a analógico, se hace la justificación de alternativas de hardware y software a partir de los requisitos, las restricciones, los cálculos y la evidencia experimental.

**Palabras clave:** sistemas embebidos, programación en c, Raspberry Pi Pico, Instrumento musical, temporización, SysTick, procesamiento de muestras.

## Alcance y operación

El sistema tiene como propósito generar siete sonidos musicales (DO, RE, MI, FA, SOL, LA, SI) donde pueden cambiarse a tres octavas diferentes. La interacción con el usuario se realizará mediante pulsadores, mediante los cuales se seleccionarán las notas musicales y se realizará el cambio entre las diferentes octavas disponibles. El sistema contará con seis pulsadores destinados a la selección de las notas y dos pulsadores para modificar la octava.

Una vez seleccionada una nota y una octava, el sistema generará la señal correspondiente. Esta señal será procesada mediante una etapa de sonido que incluye un DAC para realizar la conversión a una señal analógica y un bafle para su reproducción.

El sistema tendrá una operación monofónica, por lo que no se contempla la generación simultánea de varias notas. En consecuencia, durante la operación solo podrá reproducirse un sonido a la vez. Se asume que el sistema funcionará bajo condiciones de operación ideales y no está destinado a ambientes extremos. Adicionalmente, la alimentación del sistema será proporcionada por el computador, por lo que no se contempla una fuente de alimentación externa dentro del alcance del proyecto.

El instrumento cuenta con 4 modos. Un modo espera donde el sistema está encendido y esperando la interacción con el usuario, un modo de selección en donde se escoje la nota y su registro, un modo de reproducción en donde se ejecuta el sonido que se escoge. Esto se da de manera ciclica.

## Requisitos y especificaciones

Una vez definido el alcance y la operación del sistema, se establecen los requisitos que deberá cumplir el instrumento musical para alcanzar el funcionamiento esperado. Estos requisitos permiten transformar las necesidades identificadas en condiciones concretas y verificables, definiendo las funciones que debe realizar el sistema y las restricciones bajo las cuales deberá operar. Para cada requisito se establece una especificación que permita determinar objetivamente su cumplimiento durante las etapas de prueba y validación. Se anexa en la bibliografia una tabla de excel la cual contiene los requisitos funcionales y no funcionales propuestos.

## Pruebas y trazabilidad

Con el propósito de verificar el cumplimiento de los requisitos establecidos para el instrumento musical Octavia, se define un plan de pruebas que permita evaluar de manera objetiva su funcionamiento. Para ello, se establecen las condiciones iniciales, los estímulos aplicados, las variables que se observarán y los criterios de aceptación correspondientes a cada prueba.

La trazabilidad permitirá relacionar cada requisito con las pruebas diseñadas para verificarlo, las decisiones de diseño asociadas y las evidencias obtenidas durante la implementación y evaluación del sistema. De esta mane-

ra, será posible identificar los requisitos cumplidos, las posibles desviaciones y las limitaciones del instrumento, sustentando las conclusiones del proyecto mediante resultados reproducibles.Nuestra matriz de trazabilidad se encuentra en una tabla de excel referenciada en el apartado de bibliografias.

Las pruebas se organizan por capas, de modo que cada una aísla un bloque del sistema y se compila con su propio programa de prueba. Primero se valida la base de tiempo y el convertidor, después la lógica de los botones y, por último, la integración de los botones con la síntesis, incluyendo la medición de tiempos.

## Prueba 1: SysTick y DAC (generador de 440 Hz)

**Objetivo.** Verificar que el temporizador SysTick genera interrupciones periódicas a la frecuencia configurada y que la rutina de servicio actualiza el DAC recorriendo la tabla senoidal.

**Condiciones y estímulo.** Se carga una tabla senoidal de 64 muestras de 8 bits. El SysTick se configura con un valor de recarga de 4438, lo que da una frecuencia de muestreo de $125\,\text{MHz}/4439 \approx 28\,160\,\text{Hz}$. En cada interrupción se escribe una muestra en el DAC y se incrementa un contador de muestras. Cada segundo, el programa imprime por el puerto serie el número de muestras enviadas, los ciclos senoidales completados y la fase actual de la tabla.

**Variables observadas y criterios de aceptación.**

- Muestras enviadas por segundo: deben ser $\approx 28\,160$, sin interrupciones perdidas.
- Ciclos senoidales por segundo: con 64 muestras por ciclo deben ser $28\,160/64 = 440$, es decir, una señal de 440 Hz (La4).
- Estabilidad: el sistema debe funcionar de forma continua sin fallas.

**Evidencia.**

- El contador avanza en promedio 28 160 muestras por segundo y 440 ciclos por segundo, lo que confirma la frecuencia de muestreo y la frecuencia de la señal generada.
- La fase de la tabla se desplaza aproximadamente una posición cada dos segundos, de forma constante durante toda la captura. Esto corresponde a una deriva de unos 0,5 muestras por segundo, es decir, un error de frecuencia cercano al 0,001 %, debido a que el valor de recarga es entero.
- La rutina de interrupción propia queda instalada en la tabla de vectores y ejecuta `dac_escribir()` sin bloquear el sistema.
- El programa funcionó de manera continua durante más de 140 s sin fallas, y el contador alcanzó cerca de $3{,}97 \times 10^6$ muestras con 62 038 ciclos completados.

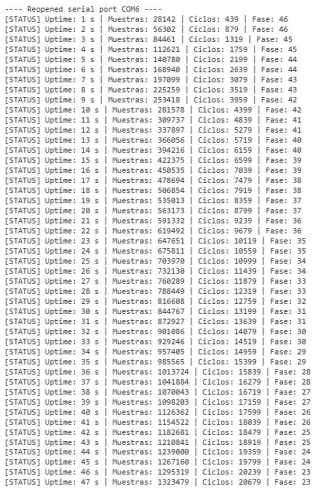



Figura **0-1**: Resultado de la Prueba 1 en el monitor serie: primeros 47 s.

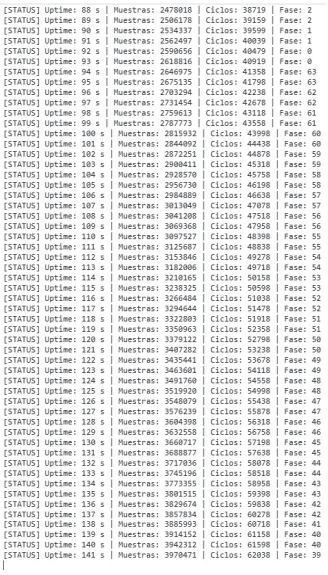



Figura **0-2**: Resultado de la Prueba 1 en el monitor serie: segundos 88 a 141.

## Prueba 2: Botones y LEDs

**Objetivo.** Verificar la lectura de los botones, la generación de eventos de pulsación y liberación, junto con la selección de octava y los LEDs indicadores. Tener en cuenta que se dice octava 0,1 y 2, no porque en la aplicacion final tengamos esas octavas, sino porque nos referimos a la posicion de los botones de octavas de izquierda a derecha en el circuito.

**Condiciones y estímulo.** Se inicializa el driver de botones y un temporizador periódico llama a `bl_actualizar()` cada 1 ms. El lazo principal extrae los eventos con `bl_notas_tomar_eventos()` y los imprime por el puerto serie junto con la octava vigente. Se pulsan y sueltan las notas de forma individual y simultánea, y se pulsan los botones de octava.

**Variables observadas y criterios de aceptación.**

- Cada pulsación genera un único mensaje de nota presionada y cada liberación un único mensaje de nota liberada, sin repeticiones por rebote.
- Cada botón corresponde al GPIO y a la nota esperados (do a si en orden).
- Al presionar varias teclas a la vez se registra un evento por cada una.
- Los botones de octava modifican la octava vigente y los LEDs muestran la octava seleccionada.

**Evidencia.**

- Se recorrieron las siete notas (do a si) en las tres octavas, y cada mensaje de nota presionada va acompañado de su mensaje de nota liberada.
- Cada botón responde con la nota esperada, lo que verifica la asignación de pines y el funcionamiento de las resistencias *pull-up* internas.
- La octava vigente impresa junto a cada nota cambia de 0 a 1 y a 2 al pulsar los botones de octava.

```
---- Opened the serial port COM6 ----
Nota do PRESIONADA (octava 0)
Nota do liberada
Nota re PRESIONADA (octava 0)
Nota re liberada
Nota mi PRESIONADA (octava 0)
Nota mi liberada
Nota fa PRESIONADA (octava 0)
Nota fa liberada
Nota fa PRESIONADA (octava 0)
Nota fa liberada
Nota sol PRESIONADA (octava 0)
Nota sol liberada
Nota la PRESIONADA (octava 0)
Nota la liberada
Nota si PRESIONADA (octava 0)
Nota si liberada
Nota do PRESIONADA (octava 1)
Nota do liberada
Nota re PRESIONADA (octava 1)
Nota re liberada
Nota mi PRESIONADA (octava 1)
Nota mi liberada
Nota fa PRESIONADA (octava 1)
Nota fa liberada
Nota sol PRESIONADA (octava 1)
Nota sol liberada
Nota la PRESIONADA (octava 1)
Nota la liberada
Nota si PRESIONADA (octava 1)
Nota si liberada
Nota do PRESIONADA (octava 2)
Nota do liberada
Nota re PRESIONADA (octava 2)
Nota re liberada
Nota re PRESIONADA (octava 2)
Nota re liberada
Nota mi PRESIONADA (octava 2)
Nota mi liberada
Nota fa PRESIONADA (octava 2)
Nota fa liberada
Nota sol PRESIONADA (octava 2)
Nota sol liberada
Nota la PRESIONADA (octava 2)
Nota la liberada
Nota si PRESIONADA (octava 2)
Nota si liberada
```

Figura **0-3**: Eventos de notas presionadas y liberadas en las octavas 0, 1 y 2 durante la Prueba 2.

## Prueba 3: Ataque, legato y tiempo de respuesta

**Objetivo.** Verificar que el antirrebote por software es de 30 ms medidos con el temporizador del sistema, medir la latencia entre la pulsación y la orden de sonido, y comprobar el comportamiento *legato* y la prioridad de notas.

**Condiciones y estímulo.** El driver guarda en la interrupción el instante del primer flanco de cada botón con `time_us_32()`. La función `bl_actualizar()` valida el botón cuando han transcurrido 30 ms desde ese instante, de modo que el antirrebote no depende del periodo del lazo principal. En `octavia_ejecutar()` se registran tres instantes: el primer flanco, la detección del evento validado y el retorno de `synth_tocar_nota()`. Con ellos se imprimen tres tiempos para cada ataque y cada legato. Se pulsan teclas individuales, se sostienen varias teclas y

se suelta una de ellas mientras otras siguen presionadas.

**Variables observadas y criterios de aceptación.**

- **Flanco a detección:** debe ser de aproximadamente 30 ms (entre 30,0 y 30,1 ms con el lazo libre). Un valor menor a 30 ms indicaría que el antirrebote no está filtrando.
- **Detección a sonido:** tiempo que tarda `synth_tocar_nota()`, esperado del orden de decenas de microsegundos.
- **Total:** latencia desde la pulsación hasta la orden de sonido, esperada cercana a 30 ms y menor al máximo de 50 ms exigido al sistema.
- **Legato:** al soltar una tecla con otras sostenidas, el sonido pasa a la nota más aguda que sigue presionada sin silencio intermedio. Solo se silencia cuando no queda ninguna tecla presionada.
- **Prioridad y octava:** si se presionan varias notas nuevas, suena la más aguda, y la octava queda fijada mientras haya teclas sostenidas.

**Evidencia.**

- Confirmación del antirrebote de 30 ms por software mediante los tiempos medidos de flanco a detección.
- Medición de la latencia total de ataque y de legato.

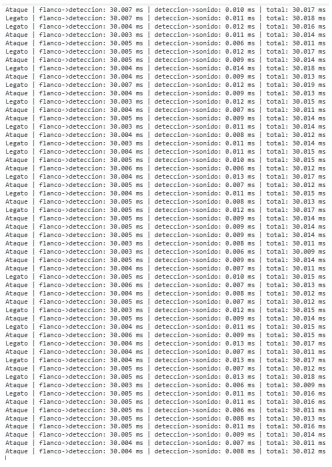



Figura **0-4**: Tiempos de ataque y legato medidos en la Prueba 3.

## Resumen de las pruebas

La Prueba 1 valida la base de tiempo y la síntesis de la señal, la Prueba 2 valida la lógica de los botones y la Prueba 3 integra los botones con la síntesis e incorpora las mediciones de tiempo.

Cabe resaltar que cada una de estas pruebas solo son para cada componente en especifico y no para el sistema en general.

# Asignación de requisitos a hardware y software

Una vez definidos los requisitos funcionales y no funcionales del instrumento musical Octavia, se establece su asignación a los componentes de hardware y a los módulos de software responsables de su cumplimiento. Esta distribución permite identificar las responsabilidades de cada elemento del sistema y las relaciones necesarias para su integración.

En el hardware se consideran los elementos de interacción con el usuario, el microcontrolador Raspberry Pi Pico basado en el RP2040, el conversor digital-analógico y la interfaz de salida de audio. Por su parte, el software comprende los módulos encargados de gestionar los controles, seleccionar las notas y octavas, generar las muestras de sonido y administrar la temporización mediante el driver propio del SysTick.

La asignación permite establecer una relación entre los requisitos y los elementos que intervienen en su cumplimiento, facilitando la planificación de las pruebas de componente, integración y aceptación. Asimismo, sirve como referencia para definir las interfaces entre los módulos y verificar que las responsabilidades de hardware y software sean coherentes con el diseño propuesto. Las especificaciones de nuestro diseño se encuentran en una tabla de excel, su enlace está en las referencias bibliograficas.

# Alternativas y decisiones

Durante el diseño del instrumento musical Octavia se evalúan diferentes alternativas de hardware y software con el propósito de seleccionar las soluciones que permitan cumplir los requisitos establecidos. A continuación, se documenta la decisión de diseño correspondiente a la etapa de conversión analógica.

## Decisión de Diseño DD-01: Selección del convertidor digital-analógico (DAC)

El requisito RF-06 y la restricción de la práctica exigen generar una señal analógica de audio compatible con un altavoz amplificado, utilizando un DAC externo que debe ser una red R-2R o un DAC de interfaz paralela.

**Criterios de evaluación y restricciones:**

- **Precisión y linealidad:** Capacidad de reproducir la onda sin distorsión armónica excesiva.
- **Costo y disponibilidad:** Facilidad de consecución de los componentes en el mercado local.
- **Uso de recursos (GPIO):** Cantidad de pines requeridos del microcontrolador RP2040.
- **Complejidad de integración:** Esfuerzo requerido para el acondicionamiento eléctrico de la señal de salida.

### Alternativa A: DAC R-2R de 8 bits con amplificador operacional

Se implementa una red de resistencias R–2R acompañada de un amplificador operacional en configuración seguidor de tensión o amplificador no inversor. El principio de funcionamiento se basa en la contribución ponderada de cada bit (8 pines GPIO en total) de la palabra digital de entrada. Entre las principales ventajas de esta solución se encuentran la posibilidad de construir el conversor con componentes discretos de muy bajo costo y su utilidad pedagógica. Sin embargo, su precisión depende críticamente de la tolerancia de las resistencias; una tolerancia comercial del 5 % introduce no linealidades significativas (ruido de cuantificación y distorsión), por lo que se requiere un emparejamiento manual o el uso de resistencias del 1 %.

### Alternativa B: DAC de interfaz paralela integrado

Se utiliza un circuito integrado DAC comercial con interfaz paralela de 8 bits, como el DAC0808. Este dispositivo recibe las muestras digitales generadas por el RP2040 mediante un bus de 8 líneas de datos. El circuito integrado garantiza una linealidad y un tiempo de establecimiento (settling time) especificados en su hoja de datos, lo cual asegura una alta fidelidad para la forma de onda del sonido. Sin embargo, requiere una fuente de alimentación dual (positiva y negativa) y circuitería externa adicional de referencia, lo que aumenta la complejidad del montaje y el costo económico en comparación con la red pasiva.

### Comparación de alternativas

Para el montaje del hardware se hace una tabla comparativa con las dos opciones de diseño. Así se busca ver de una manera más clara que ventajas y desventajas se tienen con cada implementación. La tabla se anexa en el link de bibliografia en un archivo de excel.

### Alternativa seleccionada: DAC R-2R con amplificador operacional

**Decisión y justificación:** Para el desarrollo de Octavia se seleccionó la Alternativa A (red R-2R). Aunque la Alternativa B ofrece una linealidad superior por hoja de datos, la red R-2R cumple de manera suficiente con la calidad de audio requerida si se utilizan resistencias medidas cuidadosamente. La decisión se fundamenta fuertemente en el bajo costo, la alta disponibilidad de los componentes y la facilidad de montaje al requerir únicamente alimentación simple de 3.3V o 5V para el amplificador operacional, evitando la necesidad de fuentes duales que exige el DAC0808.

**Consecuencias y compromisos asumidos:** Al elegir esta alternativa, se asume el compromiso de que un desbalance en las resistencias generará distorsión armónica. Como medida de mitigación, se medirán todas las resistencias con multímetro antes del ensamble para garantizar una tolerancia emparejada menor al 1 %. Adicionalmente, se asume el uso de 8 pines GPIO contiguos del RP2040, lo cual obliga a planificar cuidadosamente la asignación de pines restantes para los botones de las notas y las octavas.

# Esquema eléctrico

La Figura **0-6** muestra el esquema eléctrico del instrumento Octavia. El circuito se divide en cuatro bloques: la interfaz de entrada (pulsadores y LEDs), el microcontrolador, el convertidor digital-analógico R-2R y la etapa de acondicionamiento y salida de audio.

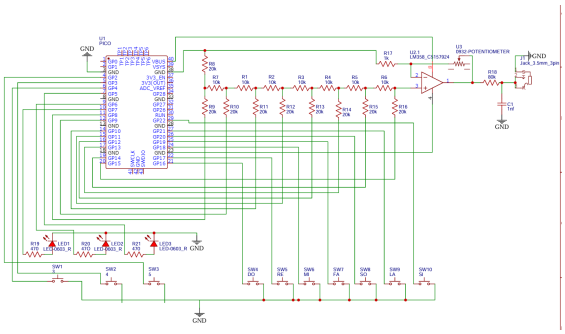


Figura **0-6**: Esquema eléctrico del instrumento Octavia.

## Alimentación

El sistema se alimenta desde el computador a través del puerto USB de la Raspberry Pi Pico. El pin VBUS (5 V) alimenta el amplificador operacional LM358, mientras que la lógica del RP2040 y las salidas GPIO operan a 3.3 V. Todas las masas del circuito (microcontrolador, red R-2R, amplificador, pulsadores, LEDs y conector de audio) están conectadas a una referencia común.

## Interfaz de entrada y visualización

Los pulsadores SW4 a SW10 corresponden a las siete notas (DO, RE, MI, FA, SOL, LA, SI), y SW1 a SW3 seleccionan la octava (3, 4 y 5). Cada pulsador conecta su pin GPIO a tierra al presionarse. Como los pines se configuran con resistencias *pull-up* internas, no se requieren resistencias externas y el nivel lógico activo es bajo.

Los LEDs LED1 a LED3 indican la octava activa. Cada uno está conectado a una salida digital a través de una resistencia limitadora de 470 Ω, lo que da una corriente aproximada de

$$I_{LED} \approx \frac{3{,}3\,\mathrm{V} - V_F}{470\,\Omega} \approx 3\,\mathrm{mA}, \tag{0-1}$$

con $V_F \approx 2\,\mathrm{V}$ para un LED rojo, valor seguro para los pines del RP2040.

## Convertidor digital-analógico R-2R

El DAC convierte la palabra digital de 8 bits calculada por el sintetizador en una tensión proporcional a su valor. Se construye únicamente con resistencias de dos valores, $R = 10\,\mathrm{k}\Omega$ y $2R = 20\,\mathrm{k}\Omega$, conectadas en forma de escalera (*ladder*), y no requiere componentes activos.

**Estructura.** Ocho pines GPIO contiguos del RP2040 forman el bus de datos. Cada pin $b_k$ (con $k = 0$ el bit menos significativo, LSB, y $k = 7$ el más significativo, MSB) se conecta a un nodo de la escalera a través de una resistencia de $2R$ (R9 a R16). Los nodos se unen entre sí mediante siete resistencias de valor $R$ en serie (R7 y R1 a R6). La resistencia R8 ($2R$) conecta a tierra el nodo del LSB y termina la escalera. La salida se toma en el nodo del MSB, que es el que se conecta al amplificador operacional.

**Principio de funcionamiento.** Cada pin GPIO entrega 0 V cuando el bit vale 0 y $V_{DD} = 3{,}3\,\mathrm{V}$ cuando vale 1. La red es lineal, por lo que se puede aplicar superposición: la tensión de salida es la suma de lo que aporta cada bit por separado, con los demás pines a 0 V. La clave de la topología es que, mirando hacia la izquierda desde cualquier nodo de la escalera, la resistencia equivalente es siempre $2R$:

- En el nodo del LSB, R8 y R9 quedan en paralelo: $2R \parallel 2R = R$. Al sumarle la resistencia serie R7 se obtiene $R + R = 2R$.
- Ese $2R$ queda en paralelo con la rama $2R$ del siguiente bit, lo que de nuevo da $R$, y al sumar la siguiente resistencia serie vuelve a obtenerse $2R$.

Este patrón se repite en los ocho nodos. Como consecuencia, en cada nodo la tensión se divide a la mitad al avanzar un paso hacia la salida: un bit aporta la mitad de lo que aporta el bit inmediatamente superior. Así, el bit $b_k$ contribuye con

$$V_k = V_{DD}\,\frac{b_k}{2^{\,8-k}}, \qquad k = 0, 1, \ldots, 7, \tag{0-2}$$

es decir, $V_{DD}/2$ para el MSB, $V_{DD}/4$ para el siguiente, y así hasta $V_{DD}/256$ para el LSB. Esta ponderación binaria es la que hace que la red sea un conversor. Sumando todos los aportes, con $N = \sum_{k=0}^{7} b_k\,2^k$ el valor decimal de la palabra digital:

$$V_{DAC} = \sum_{k=0}^{7} V_{DD}\,\frac{b_k}{2^{\,8-k}} = V_{DD}\,\frac{N}{2^8}, \qquad N \in \{0, 1, \ldots, 255\}. \tag{0-3}$$

**Resolución y rango.** El tamaño de cada escalón es

$$\Delta V = \frac{V_{DD}}{2^8} = \frac{3{,}3\,\mathrm{V}}{256} \approx 12{,}9\,\mathrm{mV}, \tag{0-4}$$

y el valor máximo, para $N = 255$, es $V_{DAC,max} = 3{,}3 \cdot \frac{255}{256} \approx 3{,}29\,\mathrm{V}$. Por ejemplo, la palabra $N = 128$ (solo el MSB en alto) produce $V_{DD}/2 = 1{,}65\,\mathrm{V}$, que corresponde al nivel medio de la señal senoidal, ya que el sintetizador centra la onda en 128. Un valor de $N = 64$ produce 0,825 V.

**Impedancia de salida.** Mirando desde el nodo de salida hacia la red, con todos los bits a 0, la resistencia

de Thévenin es siempre $R = 10\,\mathrm{k\Omega}$, sin importar la palabra digital. Esto tiene dos consecuencias. Primero, la salida no debe cargarse con una resistencia baja, porque formaría un divisor con los $10\,\mathrm{k\Omega}$ y alteraría la escala; por eso se conecta directamente a la entrada de alta impedancia del amplificador operacional. Segundo, la resistencia de salida de los pines GPIO (de decenas de ohmios) es despreciable frente a los $20\,\mathrm{k\Omega}$ de cada rama, de modo que el error que introduce es muy pequeño.

**Escritura simultánea y calidad de la señal.** Todos los bits de la muestra se escriben en un único ciclo mediante el bus SIO, de modo que la salida cambia de un escalón al siguiente sin estados intermedios. Si los bits cambiaran en instantes distintos, aparecerían picos transitorios (*glitches*) especialmente en las transiciones grandes, como de 127 a 128. Finalmente, la linealidad depende de la relación entre las resistencias: un error en la resistencia de un bit alto desplaza el nivel de salida del MSB en una cantidad mucho mayor que el escalón del LSB. Por eso se miden las resistencias antes del montaje y se exige una tolerancia menor al 1 %, tal como se estableció en la decisión DD-01.

## Amplificador operacional

La salida del DAC se conecta a la entrada no inversora del LM358 (U2.1), configurado como amplificador no inversor. La resistencia R17 ($1\,\mathrm{k\Omega}$) conecta la entrada inversora a tierra y el potenciómetro U3 forma la rama de realimentación, de modo que la ganancia es

$$G = 1 + \frac{R_{U3}}{R_{17}}, \tag{0-5}$$

donde $R_{U3}$ es la resistencia ajustada en el potenciómetro. Esto permite regular la amplitud de la señal entregada al altavoz. El amplificador también aísla la red R-2R de la carga, ya que su alta impedancia de entrada evita que la red se vea afectada por lo que se conecte a la salida.

## Filtro pasabajas de salida

La salida del amplificador pasa por un filtro RC pasivo de primer orden formado por R18 ($150\,\mathrm{k\Omega}$) en serie y C1 ($1\,\mathrm{nF}$) a tierra, antes del conector Jack de 3.5 mm.

**Justificación.** Un DAC mantiene constante cada muestra durante un periodo $T_s = 1/f_s = 25\,\mu\mathrm{s}$ (con $f_s = 40\,\mathrm{kHz}$), por lo que su salida es una señal escalonada y no una senoidal continua. Ese escalonamiento aparece como componentes de alta frecuencia no deseadas (réplicas espectrales alrededor de $f_s$ y sus múltiplos, es decir en $f_s \pm f_0$, $2f_s \pm f_0$, etc.), que se perciben como ruido rectangular superpuesto a la nota. El filtro actúa como filtro de reconstrucción: deja pasar la frecuencia fundamental de la nota y atenúa estas componentes.

**Cálculo de la frecuencia de corte.** La frecuencia de corte de un filtro RC pasabajas de primer orden se calcula como

$$f_c = \frac{1}{2\pi RC}, \tag{0-6}$$

que es la frecuencia a la cual la magnitud de la salida cae a $1/\sqrt{2}$ del valor en la banda de paso ($-3\,\mathrm{dB}$). Con $R = 150\,\mathrm{k\Omega}$ y $C = 1\,\mathrm{nF}$:

$$f_c = \frac{1}{2\pi\,(150\times10^{3}\,\Omega)(1\times10^{-9}\,\mathrm{F})} = \frac{1}{9{,}42\times10^{-4}\,\mathrm{s}} \approx 1061\,\mathrm{Hz}. \tag{0-7}$$

La nota de mayor frecuencia del instrumento es SI de la octava 5 ($f_{max} \approx 987{,}77\,\mathrm{Hz}$). La frecuencia de corte se ubica justo por encima de esa frecuencia, de modo que todas las notas quedan dentro de la banda de paso o en su borde, y a la vez se atenúan fuertemente las componentes cercanas a $f_s$. Este criterio de diseño puede expresarse como $f_{max} f_c \ll f_s$.

Con este valor de $f_c$, la nota SI de la octava 5 sufre una atenuación de 2,71 dB (su amplitud se reduce a cerca del 73 %) y un desfase de $-42{,}9°$, mientras que las notas graves casi no se modifican. La diferencia de volumen entre notas se puede compensar ajustando la ganancia del amplificador. A cambio, las réplicas espectrales cercanas a la frecuencia de muestreo se atenúan más de 31 dB.

## Salida de audio

La señal filtrada se entrega a través del conector Jack de 3.5 mm al altavoz amplificado (bafle), con el contacto de tierra del conector conectado a la referencia común del circuito.

# Componentes y Arquitectura

El diseño del instrumento Octavia se divide en una arquitectura de hardware encargada de la interfaz física y la conversión analógica, y una arquitectura de software responsable de la síntesis de audio y la temporización.

## Arquitectura de Hardware

El hardware del sistema tiene como núcleo de procesamiento el microcontrolador RP2040 en la placa Raspberry Pi Pico. El sistema se compone de tres bloques principales:

- **Interfaz de entrada:** Compuesta por pulsadores mecánicos para la selección de las siete notas musicales (do, re, mi, fa, sol, la, si) y controles adicionales para el cambio de octava. Estos botones se conectan a pines GPIO configurados con resistencias de *pull-up* internas.
- **Unidad de procesamiento:** El RP2040 adquiere el estado de los controles, determina la frecuencia fundamental correspondiente y calcula el valor digital de la muestra de audio en tiempo real.
- **Etapa de salida analógica:** Conformada por una red resistiva R-2R de 8 bits acoplada a los pines GPIO del microcontrolador. La salida de la red pasiva se conecta a un amplificador operacional en configuración de seguidor de tensión para acoplar la impedancia y entregar la señal analógica al altavoz amplificado.

# Arquitectura de Software

Arquitectura de software

El software se estructura bajo un paradigma híbrido que combina un lazo principal de sondeo **polling** con una rutina asíncrona basada en interrupciones. Esta arquitectura permite separar las tareas de interacción con el usuario de las tareas críticas de síntesis de audio, atendiendo a los requisitos de temporización del sistema.

Para la generación de sonido se emplea el método de síntesis digital directa **Direct Digital Synthesis, DDS**, implementado mediante una tabla de ondas que almacena muestras de una señal senoidal. Este método permite controlar la frecuencia de las notas mediante un acumulador de fase, por lo que resulta adecuado para la generación de señales de frecuencia definida en instrumentos musicales digitales.

El principio de funcionamiento se basa en mantener una frecuencia de muestreo constante, determinada por la configuración del SysTick, mientras se modifica la velocidad de recorrido de la tabla para obtener las diferentes frecuencias musicales. De esta manera, el temporizador se configura una sola vez y no requiere modificaciones para cambiar de nota.

Para cada nota se establece un incremento de fase que se suma al acumulador en cada interrupción del SysTick. El valor acumulado determina la posición de lectura dentro de la tabla de ondas y, por tanto, la velocidad con la que se recorre. Cuando el acumulador alcanza su límite numérico, se produce un desbordamiento que permite continuar el ciclo de forma periódica. Así, mediante la selección del incremento de fase correspondiente, se generan diferentes frecuencias sin modificar la frecuencia de muestreo del sistema.

## Capa de Aplicación: Interacción mediante Polling

El lazo principal (*super-loop*) del programa en C se encarga exclusivamente de interactuar con las entradas del usuario mediante la técnica de *polling*. De forma cíclica, el microcontrolador lee el estado de los pines GPIO asociados a los botones de las notas (do a si) y a los controles de cambio de octava.

Dentro de este lazo se implementan los mecanismos lógicos de antirrebote y se aplican las reglas de retención de octavas para las notas accionadas. Al detectar una pulsación válida, la aplicación no genera el sonido directamente, sino que actualiza variables globales compartidas, como el incremento de fase correspondiente a la frecuencia de la nota detectada. Delegar esta lectura al *polling* es una decisión de diseño justificada, ya que el tiempo de respuesta exigido (máximo de 50 ms) es lo suficientemente holgado como para ser atendido por el lazo principal sin requerir interrupciones externas por hardware en cada pin.

## Capa de Temporización y Síntesis: Interrupciones (SysTick)

Para garantizar que el sonido del instrumento no presente interrupciones no deseadas, ruido o chasquidos, la actualización del convertidor digital-analógico requiere un determinismo temporal estricto. Por ello, la síntesis de audio se independiza del lazo de *polling* y se delega a las interrupciones del temporizador del sistema (SysTick).

## Capa de salida de audio: DAC

Orientada a la conversión digital-analógica mediante un DAC de arquitectura R-2R de 8 bits. A nivel de software, las muestras generadas por el algoritmo de síntesis se acondicionan y cuantifican al rango de 8 bits, para posteriormente ser enviadas a los pines GPIO configurados como salidas digitales de la Raspberry Pi Pico. El DAC R-2R recibe estas señales digitales y realiza su conversión a niveles de tensión analógicos, permitiendo obtener la señal correspondiente a la nota musical sintetizada.

Se desarrolló un driver propio mediante el acceso directo a los registros del SysTick para configurar la frecuencia de muestreo de la señal. En cada desbordamiento del temporizador, se dispara la rutina de servicio de interrupción (`SysTick_Handler`). Esta rutina ejecuta el

algoritmo de Síntesis Digital Directa: suma el incremento de fase (actualizado por el lazo principal) al acumulador de fase, utiliza los bits más significativos para indexar una tabla de ondas senoidales precalculada y escribe el valor digital resultante en el puerto GPIO del DAC.

**Justificación de la arquitectura híbrida:** Esta combinación de interrupciones y *polling* asegura que la carga de procesamiento producida por presionar múltiples botones simultáneamente no interfiera con la regularidad temporal de la actualización de las muestras de sonido. El SysTick interrumpe periódicamente el *polling* para actualizar el DAC y retorna el control al lazo principal, garantizando una frecuencia de muestreo constante y, por consiguiente, la correcta afinación del instrumento.

# Funcionamiento de los drivers

La capa de controladores de dispositivo (*drivers*) del sintetizador *Octavia* actúa como la interfaz de abstracción entre el hardware físico del microcontrolador RP2040 (Raspberry Pi Pico) y la lógica de generación de audio en tiempo real.

## Driver del Temporizador del Sistema (`systick.c / systick.h`)

El módulo `systick` gestiona el temporizador decreciente de 24 bits integrado en el núcleo ARM Cortex-M0+. Su propósito principal es actuar como el reloj maestro de tiempo real que determina la frecuencia de muestreo física ($F_{\text{muestreo}} = 40$ kHz) del motor de síntesis DDS.

El controlador interactúa directamente con los registros de memoria de bajo nivel del procesador mediante operaciones *bare-metal*:

- **Configuración del periodo:** Calcula y escribe en el registro de recarga `SYST_RVR` el número de ciclos de reloj de CPU requeridos por muestra (reload $= \frac{f_{\text{cpu}}}{F_{\text{muestreo}}} - 1$).
- **Gestión de prioridad y registros:** Modifica el byte alto del registro de prioridad de excepciones del SCB (`SCB_SHPR3`) para asignar al SysTick una prioridad nivel `0x80`, equiparándola con la prioridad de las interrupciones GPIO para evitar anidamientos indeseados.
- **Control del temporizador:** Habilita el contador, la fuente de reloj del sistema (`clk_sys`) y la excepción de hardware mediante el registro de estado y control `SYST_CSR`.

## Driver de Botones e Indicadores LED (`buttons_leds.c / buttons_leds.h`)

El módulo `buttons_leds` gestiona 10 entradas digitales (7 notas musicales y 3 selecciones de octava) y 3 salidas digitales para la indicación visual de la octava activa. Implementa una arquitectura híbrida basada en interrupciones por hardware y filtrado por temporización no bloqueante.

El funcionamiento del controlador se divide en tres fases principales:

- **Detección por Interrupción (ISR):** Los pines de entrada están configurados en modo *pull-up* activo bajo, registrando interrupciones por flanco de subida y bajada (`IO_IRQ_BANK0`). Cuando un botón cambia de estado, la función `gpio_isr()` deshabilita inmediatamente las interrupciones del pin correspondiente para silenciar el ruido mecánico inicial, guarda el instante del primer flanco con `time_us_32()` y marca el bit del botón en una máscara de estados pendientes (`pendientes`).
- **Filtrado Antirrebote (*Debounce*):** La función `bl_actualizar()` evalúa periódicamente la máscara de pendientes. Para cada botón en espera, comprueba si han transcurrido `BL_ANTIRREBOTE_MS = 30` ms desde el instante del primer flanco, medidos con el temporizador del sistema, de modo que el tiempo de antirrebote no depende del periodo del lazo principal. Una vez confirmado el nivel lógico estable del pin, actualiza el estado consolidado (`estado_estable`), registra el evento en las máscaras de salida (`eventos_presionado` o `eventos_liberado`), limpia los flancos latentes en el hardware y habilita de nuevo la interrupción del pin.
- **Lectura Atómica y Selección de Octava:** La función `bl_notas_tomar_eventos()` extrae los eventos acumulados dentro de una sección crítica (`save_and_disable_interrupts()`), garantizando la consistencia de los datos en memoria ante posibles condiciones de carrera con las interrupciones.

## Driver del Convertidor Digital-Analógico (`dac.c / dac.h`)

El controlador `dac` abstrae la interfaz de salida de datos de 8 bits conectada externamente al circuito R-2R. Para minimizar el desfase temporal (*jitter*) en la reconstrucción analógica de la forma de onda, el controlador

opera sobre el bus de entradas/salidas de ciclo único (*SIO*) del RP2040.

Mediante el uso de máscaras de bits (`DAC_MASCARA`), las funciones del módulo realizan operaciones paralelas simultáneas:

- **Inicialización por máscara:** `dac_init()` asigna la dirección de salida y habilita simultáneamente los 8 pines del bus GPIO utilizando la función `gpio_init_mask()` del SDK.
- **Escritura atómica de muestras:** La función `dac_escribir()` toma el valor digital de 8 bits calculado dentro de la rutina `isr_systick()`, lo desplaza al pin base del bus (`DAC_PIN_BASE`) y actualiza el puerto en un solo ciclo de reloj empleando la instrucción `gpio_put_masked()`. Esto asegura que todos los bits de la muestra cambien de estado lógico exactamente en el mismo instante de tiempo.

## Módulo de Gestión de Aplicación (`octavia.c / octavia.h`)

El módulo `octavia` constituye la capa superior de control del sistema. Gestiona el flujo de estados de las notas musicales y traduce las interacciones del usuario registradas en el teclado físico hacia llamadas del motor de audio.

- **Gestión de sonido y Legato:** Mantiene un sonido limpio mientras se oprime el botón correspondiente a una nota musical. Si el usuario toca una nueva tecla mientras sostiene otra, actualiza la afinación en tiempo real aplicando la técnica de ejecución *legato*.
- **Procesamiento de Eventos:** En cada ciclo del bucle principal (`main`), la función `octavia_ejecutar()` invoca la captura de eventos del teclado (`bl_notas_tomar_eventos()`). Ante un evento de pulsación, consulta la octava seleccionada y llama a `synth_tocar_nota()`; ante un evento de liberación, evalúa si quedan notas sostenidas en la máscara para conmutar la frecuencia o silenciar la salida mediante `synth_liberar_nota()`.

## Módulo de Generación de Audio y Síntesis DDS (`synth.c / synth.h`)

El módulo `synth` implementa el motor de síntesis digital directa y el control de envolvente de amplitud en tiempo real. Es el responsable de computar cada muestra de audio enviada al convertidor D/A.

- **Tabla de Onda Senoidal Discreta (`tabla_onda`):** Contiene una tabla de búsqueda (*Look-Up Table* o LUT) fija de 256 muestras de 8 bits que representan un período completo de una señal senoidal pura, precalculada y cuantizada en el rango de 0 a 255.
- **Acumulador de Fase de 32 bits:** La rutina de interrupción del SysTick (`isr_systick()`) incrementa en cada muestra un acumulador de 32 bits mediante un valor de fase prefijado (`incremento_fase`). Los 8 bits más significativos del acumulador se utilizan como el índice directo de lectura sobre la tabla senoidal.
- **Procesamiento de Muestra y Envolvente:** Dentro de la rutina de interrupción, la muestra leída de la tabla se centra en el origen de voltaje (muestra $-$ 128) para eliminar el *offset* de corriente continua (DC). Luego, se multiplica por la variable `nivel_envolvente` para ajustar la amplitud o decaimiento del sonido, y finalmente se re-desplaza a un entero sin signo antes de transmitirse al driver del DAC.

## Módulo de Cálculo de Frecuencias y Notas (`notes.c / notes.h`)

El módulo realiza el mapeo matemático entre las notas musicales de la escala temperada igual y los incrementos de fase requeridos por el algoritmo.

- **Tabla de Búsqueda Precalculada:** En lugar de ejecutar funciones de coma flotante complejas dentro de la rutina de interrupción, la función `notes_init()` precalcula durante el arranque del sistema una matriz bidimensional con los valores exactos de incremento de fase para 3 octavas completas aplicando la fórmula:

$$\text{inc} = \frac{f_{\text{nota}} \cdot 2^{32}}{f_{\text{real}}} \qquad \text{(0-8)}$$

donde $f_{\text{real}}$ es la frecuencia de muestreo exacta devuelta por la configuración del hardware (40 kHz).

- **Afinación Dinámica:** La función `notes_get_incremento(octava, nota)` entrega instantáneamente en tiempo constante, el incremento de fase necesario para actualizar el registro del sintetizador.

# Calibración

Con el objetivo de verificar la afinación del instrumento y el cumplimiento del margen de error máximo del 1 %, se calcularon los incrementos de fase correspondientes a las 21 notas soportadas por el sistema. Posteriormente, las señales generadas se evaluaron mediante un osciloscopio y un afinador, con el propósito de verificar su comportamiento y determinar las frecuencias obtenidas.

Para cada nota se registró la forma de onda y se midió su frecuencia, la cual se comparó con la frecuencia teórica de referencia para calcular el error relativo. Los registros de las señales medidas se presentan en la carpeta de drive [1], mientras que los valores de frecuencia y los errores calculados se encuentran en la hoja ≪Tabla de frecuencias≫ del siguiente excel [2].

Los resultados muestran que el error relativo de frecuencia se mantiene dentro del límite establecido del 1 % para las 21 notas evaluadas. De esta manera, se verifica el cumplimiento del requisito no funcional **RNF-01** en lo referente a la precisión de las frecuencias generadas.

# Resultados de componentes

El funcionamiento del sistema octavia, en el que se reproducen las 21 notas que lo integran se muestra en el siguiente video, Link.

# Conclusiones

- **Determinismo Temporal y Respuesta en Tiempo Real:** La implementación del temporizador hardware `SysTick` del microcontrolador ARM Cortex-M0+ permitió asegurar un periodo de interrupción estrictamente periódico de 25 $\mu$s (40 kHz). Esta arquitectura garantiza que la generación de muestras de audio por síntesis directa de frecuencia (DDS) ocurra de manera determinista, eliminando el *jitter* temporal y evitando distorsiones por fluctuación en la señal analógica reconstruida.
- **Eficiencia en el Procesamiento Matemático (DDS):** La precomputación de los incrementos de fase en una tabla de búsqueda en tiempo de arranque, sumada al uso de un acumulador de fase de 32 bits, demostró ser una estrategia óptima para la arquitectura de 32 bits del RP2040. Esto eliminó la necesidad de ejecutar cálculos flotantes complejos dentro de la rutina de interrupción (`ISR`), permitiendo que la generación de sonido requiera un tiempo de ejecución mínimo por muestra.
- **Gestión Robusta de Entradas y Filtrado de Ruido Mechanical:** La combinación de interrupciones por flanco en hardware junto con temporización no bloqueante logró silenciar de forma efectiva el ruido mecánico (*bouncing*) de las teclas. Además, el aislamiento del procesamiento mediante secciones críticas atómicas (`save_and_disable_interrupts`) garantizó la integridad de los datos transmitidos entre las interrupciones y el bucle principal, previniendo condiciones de carrera.
- **Transferencia Paralela de Datos a Alta Velocidad:** El uso de las instrucciones por máscara del bus SIO del RP2040 en el *driver* del DAC (`dac.c`) permitió escribir muestras completas de 8 bits en un único ciclo de reloj. Esto garantizó que las entradas digitales del convertidor DAC0830LCN conmuten de manera estrictamente simultánea, preservando la continuidad de la forma de onda de salida.
- **Diseño Modular y Mantenibilidad:** La separación en tres capas claras (aplicación, motor de síntesis y controladores de bajo nivel) facilitó la depuración independiente de cada módulo y abre la posibilidad de expandir las capacidades del sintetizador en el futuro (como incluir nuevas formas de onda, envolventes ADSR completas o efectos de audio) sin alterar la arquitectura base de los controladores de hardware.

# Bibliografía


[1] Universidad de Antioquia. Formas de onda de las notas - proyecto octavia, 2026. URL https://drive.google.com/drive/folders/14WvxnDHvfCLiCVjHNGkABw-XYggMfdI-?usp=drive_link. Consultado el 19 de octubre de 2026.

[2] Universidad de Antioquia. Matriz de trazabilidad y pruebas de aceptación - proyecto digitales iii, 2026. URL https://docs.google.com/spreadsheets/d/1vgRmu98wthezT-wXwT1z1Q125rbmcB2MoGa6_3-FGSI. Consultado el 27 de septiembre de 2026.

[3] Raspberry Pi Ltd. *RP2040 Datasheet: A microcontroller by Raspberry Pi*, 2021. URL https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf. Accedido: 2026.

[4] Raspberry Pi Ltd. *Raspberry Pi Pico C/C++ SDK: Libraries and tools for C/C++ development on RP2040 microcontrollers*, 2023. URL https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf. Accedido: 2026.